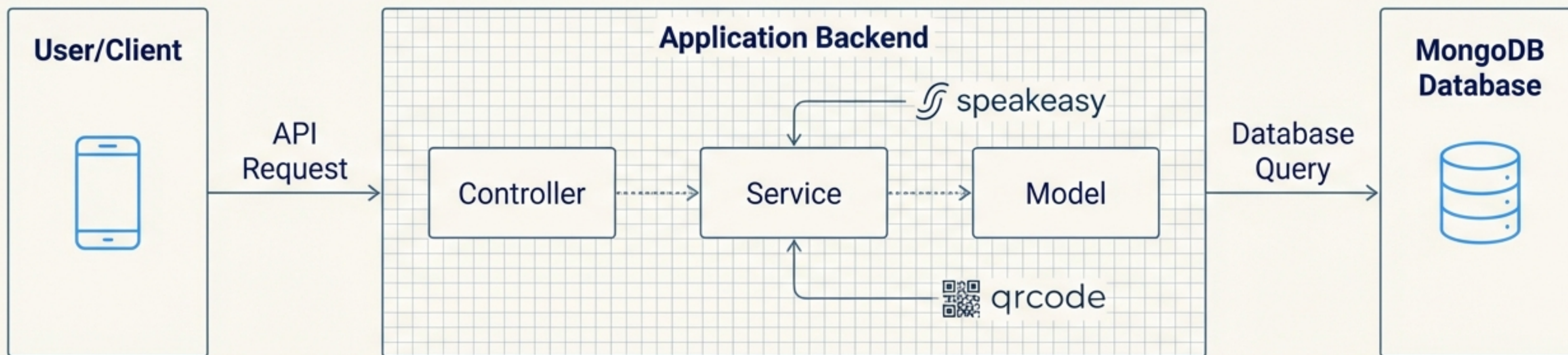


Kiến trúc Xác thực Hai Lớp (2FA)

Phân tích sâu về Hệ thống Xác thực dựa trên TOTP



Tiêu chuẩn Mở: Tuân thủ **RFC 6238**, tương thích với *Google Authenticator, Authy, Microsoft Authenticator*.



Bảo mật Tăng cường: Bổ sung lớp bảo vệ *"Something you have"* (thiết bị) vào quy trình đăng nhập.



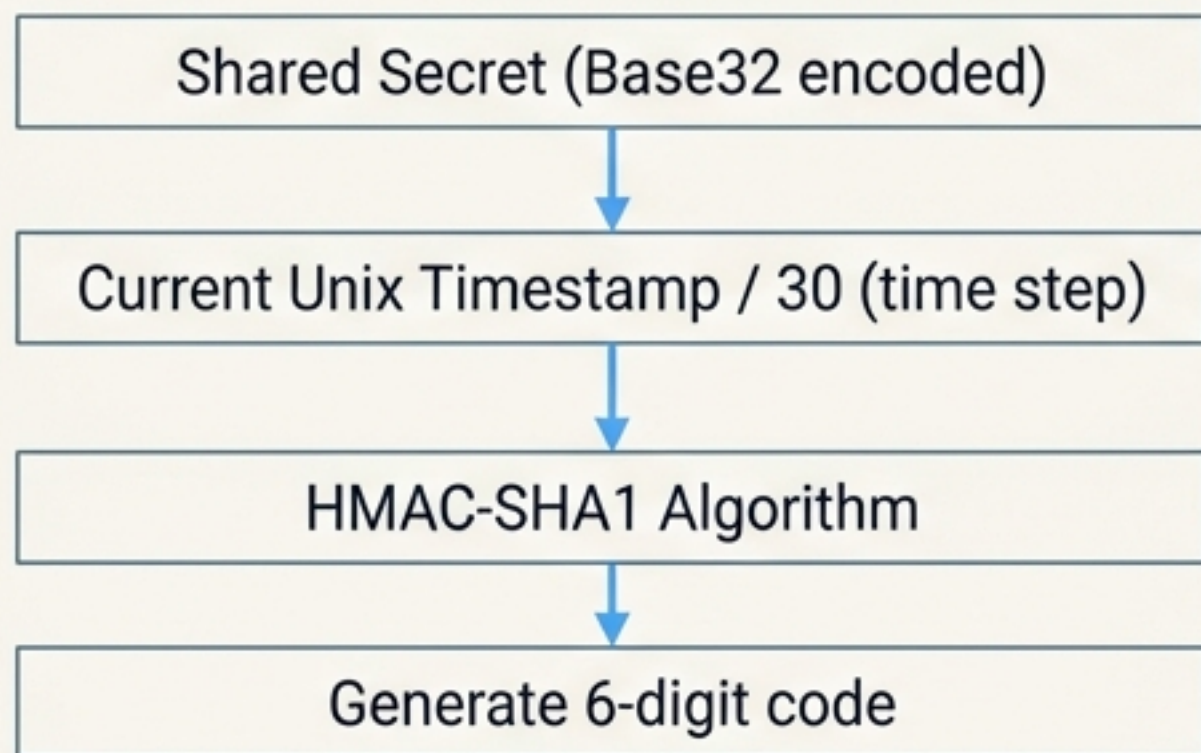
Phục hồi Tài khoản: Tích hợp sẵn **Mã dự phòng (Backup Codes)** để khôi phục quyền truy cập.

Nền tảng Công nghệ: Time-based One-Time Password (TOTP)

Định nghĩa

TOTP là một thuật toán tạo mã xác thực một lần (6 chữ số) dựa trên thời gian hiện tại. Mã này thay đổi mỗi 30 giây và được tính toán độc lập trên cả client (app xác thực) và server bằng cách sử dụng chung một Shared Secret.

Cơ chế Hoạt động



Key Parameter:

window: 2 - Cho phép sai lệch ± 60 giây để bù trừ cho lệch đồng hồ và độ trễ mạng.

Ưu điểm Chính



Hoạt động Offline: Không yêu cầu thiết bị người dùng phải có kết nối mạng.



Tự động Hết hạn: Mã chỉ có hiệu lực trong một khoảng thời gian ngắn (~30-90s).



Tiêu chuẩn Hóa: Tương thích rộng rãi với nhiều ứng dụng xác thực phổ biến.



Bảo mật: Secret key không bao giờ được truyền qua mạng trong quá trình xác thực.

Lớp Model - Nền móng Dữ liệu (`User.js`)

Cấu trúc dữ liệu 2FA được tích hợp trực tiếp vào schema của `User` để đảm bảo tính toàn vẹn và quản lý tập trung.

```
// User.js Schema
totpSecret: {
  type: String,
  select: false // <-- [Annotation: Ngăn lộ thông tin nhạy cảm trong các truy vấn mặc định.]
}
totpEnabled: {
  type: Boolean,
  default: false // <-- [Annotation: Mặc định 2FA bị tắt, yêu cầu người dùng kích hoạt.]
}
totpBackupCodes: [{
  code: {
    type: String,
    select: false // <-- [Annotation: Mã dự phòng cũng được bảo vệ.]
  },
  used: {
    type: Boolean,
    default: false // <-- [Annotation: Theo dõi trạng thái sử dụng của từng mã.]
  }
}]
```

Bảng Tóm tắt Đặc điểm Kỹ thuật

Field	Mô tả	Chi tiết Bảo mật
totpSecret	Chuỗi secret 32 ký tự (Base32)	select: false yêu cầu truy vấn tường minh, ví dụ: .select('+totpSecret').
totpEnabled	Cờ boolean để bật/tắt 2FA	Xác định luồng đăng nhập yêu cầu bước xác thực thứ hai.
totpBackupCodes	Mảng 10 mã phục hồi	Mỗi mã gồm 8 ký tự hex, chỉ sử dụng một lần (used: true).

Lớp Service - Logic Nghiệp vụ Cốt lõi (`totpService.js`)

`totpService` đóng gói toàn bộ logic nghiệp vụ liên quan đến TOTP, sử dụng các thư viện `speakeasy` và `qrcode`.



Phần 1: Khởi tạo và Sinh mã

1. `generateSecret(email, appName)`

Mục đích: Tạo `secret` và `otpauthUrl` cho người dùng.

Output: `{ secret: "JBSWY3D...", otpauthUrl: "otpauth://..." }`

2. `generateQRCode(otpauthUrl)`

Mục đích: Chuyển đổi `otpauthUrl` thành hình ảnh QR code (data URL base64) để người dùng quét.

Output: `"data:image/png;base64,iVBORw0..."`



3. `generateBackupCodes(count)`

Mục đích: Tạo một mảng gồm 10 mã phục hồi duy nhất.

Chi tiết: Mỗi mã được tạo từ `crypto.randomBytes(4)` (entropy 32-bit), đảm bảo tính ngẫu nhiên và an toàn.

Output: `[{ code: "A3F5B2C1", used: false }, ...]`

Lớp Service - Logic Nghiệp vụ Cốt lõi (`totpService.js`)

Phần 2: Xác thực và Kiểm tra



4. `verifyToken(token, secret)`

Mục đích: Xác thực mã TOTP 6 chữ số từ người dùng.

Logic Cốt lõi:

```
speakeasy.totp.verify({
  secret: secret,
  encoding: 'base32',
  token: token,
  window: 2 // Cho phép sai lệch ±60 giây
})
```

Tại sao `window = 2`?

Cải thiện UX bằng cách bù trừ cho (1) Lệch đồng hồ thiết bị, (2) Độ trễ người dùng nhập liệu, và (3) Độ trễ mạng.

5. `verifyBackupCode(inputCode, backupCodes)`

Mục đích: Xác thực mã dự phòng và đánh dấu là đã sử dụng.

1. Chuẩn hóa `inputCode` (xóa gạch nối, viết hoa).
2. Tìm mã tương ứng trong mảng `backupCodes` có trạng thái `used: false`.
3. Nếu tìm thấy, đổi trạng thái thành `used: true` và trả về mảng đã cập nhật.
4. Trả về lỗi nếu mã không tồn tại hoặc đã được sử dụng.

6. `formatBackupCodes(backupCodes)`

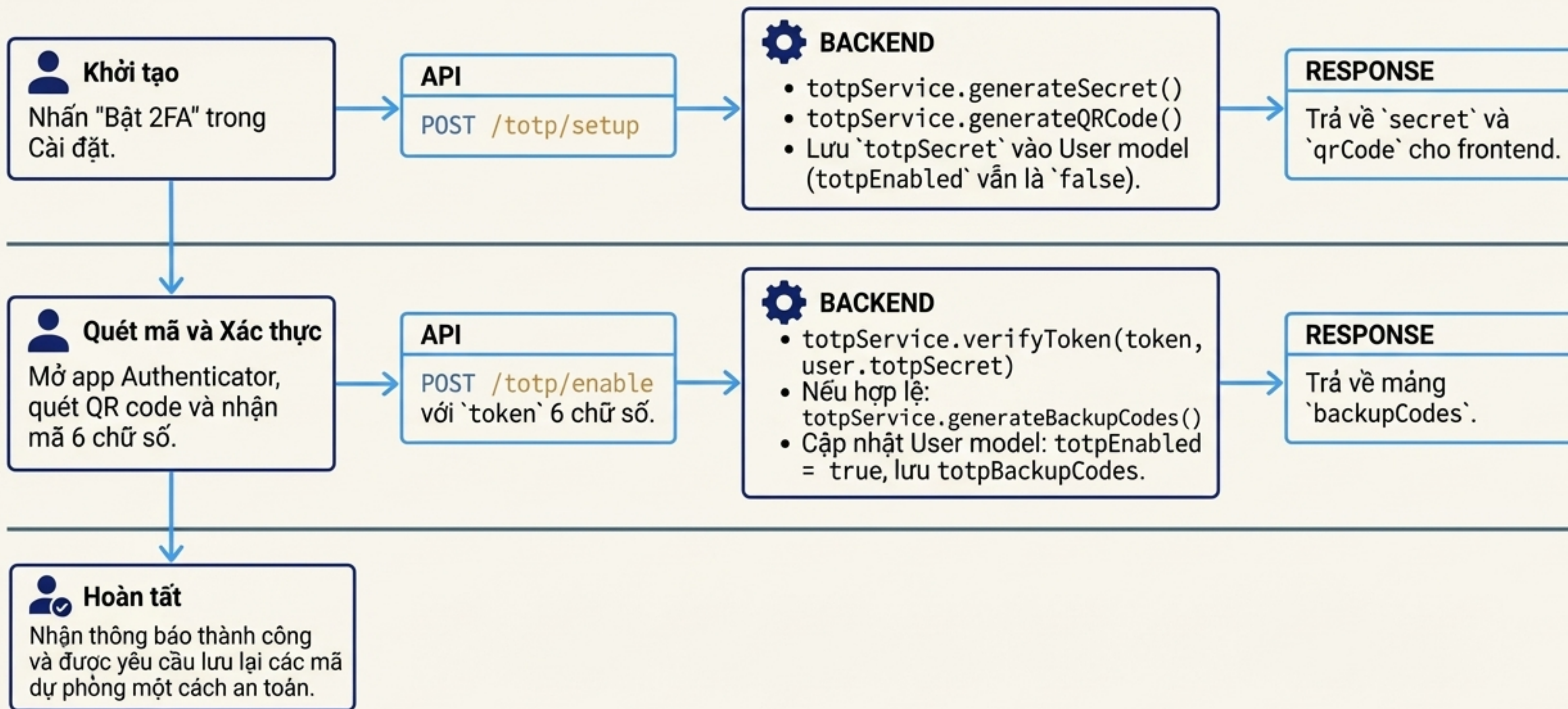
Mục đích: Định dạng mã dự phòng thành dạng `XXXX-XXXX` để người dùng dễ đọc và nhập liệu hơn.

Lớp Controller & Routing - Giao diện API của Hệ thống 2FA

Các endpoint được phân chia rõ ràng thành hai nhóm: các route công khai (public) cho quá trình đăng nhập và các route được bảo vệ (protected) để quản lý cài đặt 2FA.

Method	Endpoint	Yêu cầu Auth (JWT)	 Chức năng Chính
 POST	/api/auth/verify-totp	✗	Public: Xác thực mã TOTP trong bước 2 của luồng đăng nhập.
 POST	/api/auth/verify-backup-code	✗	Public: Xác thực mã dự phòng khi người dùng mất quyền truy cập authenticator.
 GET	/api/auth/totp/status	✓	Protected: Kiểm tra trạng thái 2FA hiện tại của người dùng (bật/tắt).
 POST	/api/auth/totp/setup	✓	Protected: Khởi tạo quá trình cài đặt, sinh <i>secret</i> và <i>QR code</i> .
 POST	/api/auth/totp/enable	✓	Protected: Kích hoạt 2FA sau khi xác thực thành công mã đầu tiên.
 POST	/api/auth/totp/disable	✓	Protected: Vô hiệu hóa 2FA (yêu cầu xác thực mã hiện tại để hoàn tất).

Hành trình Người dùng: Luồng Kích hoạt 2FA



****Lưu ý Bảo mật Quan trọng**:** Mã dự phòng chỉ được hiển thị **MỘT LẦN DUY NHẤT** tại bước này.

Hành trình Người dùng: Đăng nhập và Phục hồi Tài khoản



Luồng Đăng nhập với TOTP



1. Login (Password): POST `/login` thành công, server trả về ``requiresTOTP: true`` và ``userId``.



2. Nhập mã TOTP: Người dùng nhập mã 6 chữ số từ app xác thực.



3. Xác thực: POST `/verify-totp` với ``userId`` và ``token``.



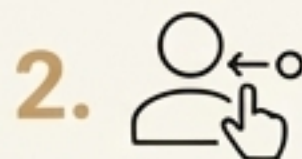
4. Thành công: Server xác thực mã, tạo JWT (Access & Refresh tokens) và cho phép đăng nhập. ✓



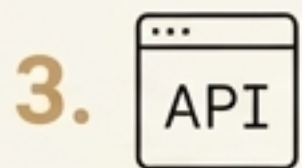
Luồng Phục hồi với Mã dự phòng



1. Login (Password): Tương tự, server trả về ``requiresTOTP: true``.



2. Chọn Phục hồi: Người dùng chọn "Sử dụng mã dự phòng".



3. Xác thực: POST `/verify-backup-code` với ``userId`` và ``backupCode``.



4. Thành công: Server xác thực mã, **đánh dấu mã đó là đã sử dụng** (`used: true`). Tạo JWT và cho phép đăng nhập.



Cảnh báo: Bạn đã sử dụng X trên 10 mã dự phòng. ✓

Các Lớp Phòng thủ và Cân nhắc về Bảo mật

Một hệ thống 2FA mạnh mẽ không chỉ nằm ở thuật toán mà còn ở các biện pháp bảo vệ xung quanh nó.



1. Các Biện pháp Hiện tại



Lưu trữ Secret

`totpSecret` và `totpBackupCodes` được bảo vệ bằng `select: false`, ngăn chặn việc vô tình bị lộ qua API.



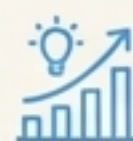
Đồng bộ Thời gian

`window: 2` ($\pm 60s$) trong `verifyToken` giúp hệ thống chống lại sự sai lệch nhỏ về thời gian giữa client và server.



Mã dự phòng Dùng một lần

Logic `verifyBackupCode` đảm bảo mỗi mã chỉ có thể được sử dụng một lần, ngăn chặn tấn công phát lại (replay attacks).



2. Đề xuất Nâng cao (Recommendations)

Rate Limiting

Triển khai giới hạn số lần thử xác thực (ví dụ: 5 lần thất bại trong 5 phút) trên các endpoint `verify-totp` và `verify-backup-code` để chống lại tấn công brute-force.

Mã hóa Dữ liệu

Hash các mã dự phòng trước khi lưu vào database để tăng cường bảo vệ trong trường hợp rò rỉ dữ liệu.

Ghi nhật ký (Audit Logging)

Ghi lại các sự kiện bảo mật quan trọng (bật/tắt 2FA, xác thực thất bại, sử dụng mã dự phòng) để phục vụ cho việc giám sát và điều tra.

Đảm bảo Chất lượng qua Kiểm thử Toàn diện

Chất lượng và độ tin cậy của hệ thống được đảm bảo thông qua một chiến lược kiểm thử đa lớp.



Kiểm thử Đơn vị (Unit Tests)

Mục tiêu: Xác minh tính đúng đắn của từng hàm riêng lẻ trong `totpService`.

- `generateSecret` trả về secret có định dạng Base32 hợp lệ.
- `verifyToken` chấp nhận mã hợp lệ và từ chối mã không hợp lệ hoặc đã hết hạn.
- `verifyBackupCode` đánh dấu chính xác một mã là đã sử dụng.

```
// Test Case Example
test('verifyToken accepts valid code within the time window', () => {
  const secret = 'JBSWY3DPEHPK3PXP'; // String in ochre
  const token = generateCurrentToken(secret); // Helper function
  expect(totpService.verifyToken(token, secret)).toBe(true);
});
```



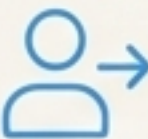
Kiểm thử Tích hợp (Integration Tests)

Mục tiêu: Xác minh sự phối hợp giữa các lớp (Controller, Service, Model) thông qua các luồng API hoàn chỉnh.

- **Luồng Cài đặt:** Mô phỏng việc một người dùng từ trạng thái chưa có 2FA, gọi `setup`, sau đó gọi `enable` thành công và nhận lại mã dự phòng.
- **Luồng Đăng nhập:** Mô phỏng việc `login` trả về `requiresTOTP`, sau đó `verify-totp` thành công và trả về access token.

Tối ưu Trải nghiệm và Hướng dẫn Xử lý sự cố

Cân nhắc về Trải nghiệm Người dùng (UX)

 **Onboarding:** Cung cấp hướng dẫn từng bước rõ ràng, tùy chọn nhập secret thủ công bên cạnh QR code.

 **Nhấn mạnh Mã dự phòng:** Hiển thị cảnh báo lớn, rõ ràng về tầm quan trọng của việc lưu trữ mã dự phòng an toàn.

Thông báo Lỗi Thân thiện

Thay vì (❌)	Nên dùng (✅)
`Invalid verification code`	"Mã bạn nhập không chính xác hoặc đã hết hạn. Vui lòng thử lại."
`Backup code used`	"Mã dự phòng này đã được sử dụng. Vui lòng thử một mã khác."

Hướng dẫn Xử lý sự cố Phổ biến

- Vấn đề:** Mã xác thực luôn báo 'Không hợp lệ'.
-  **Nguyên nhân:** Đồng hồ trên thiết bị của người dùng không được đồng bộ.
 -  **Giải pháp:** Hướng dẫn người dùng kiểm tra cài đặt 'Ngày & Giờ' và bật 'Tự động'.

- Vấn đề:** Không thể quét mã QR.
-  **Nguyên nhân:** Chất lượng hình ảnh kém, độ sáng màn hình thấp.
 -  **Giải pháp:** Cung cấp tùy chọn hiển thị secret key dạng văn bản để người dùng có thể nhập thủ công.

Tổng kết Kiến trúc và Lộ trình Phát triển



Tóm tắt Hệ thống Hiện tại

- ✓ Một hệ thống 2FA hoàn chỉnh, tuân thủ tiêu chuẩn **RFC 6238**.
- ✓ Cung cấp luồng cài đặt và sử dụng thân thiện qua **QR code**.
- ✓ Tích hợp cơ chế **phục hồi tài khoản** an toàn qua mã dự phòng sử dụng một lần.
- ✓ Thiết kế API rõ ràng, phân tách giữa các chức năng quản lý và xác thực.



Lộ trình và Các Bước Tiếp theo



Ưu tiên Cao (Bảo mật):

1. **Triển khai Rate Limiting:** Chống tấn công brute-force.
2. **Thêm Audit Logging:** Tăng cường khả năng giám sát an ninh.
3. **Hash Mã dự phòng:** Tăng cường bảo vệ dữ liệu.



Cải tiến Tương lai (UX & Tính năng):

4. **Hỗ trợ WebAuthn/FIDO2:** Cho phép sử dụng khóa bảo mật vật lý (YubiKey) và sinh trắc học.
5. **Tính năng 'Thiết bị Tin cậy':** Cho phép người dùng bỏ qua 2FA trên các thiết bị đã được xác minh trong 30 ngày.
6. **Xây dựng cơ chế Phục hồi Nâng cao:** Triển khai quy trình khôi phục tài khoản khi người dùng mất cả authenticator và mã dự phòng.